WHITEPAPER

Contextrail

COA: Context-Optimized Architecture

By Igor Titov

AI-FIRST SOFTWARE DELIVERY.

PARALLEL DELIVERY.

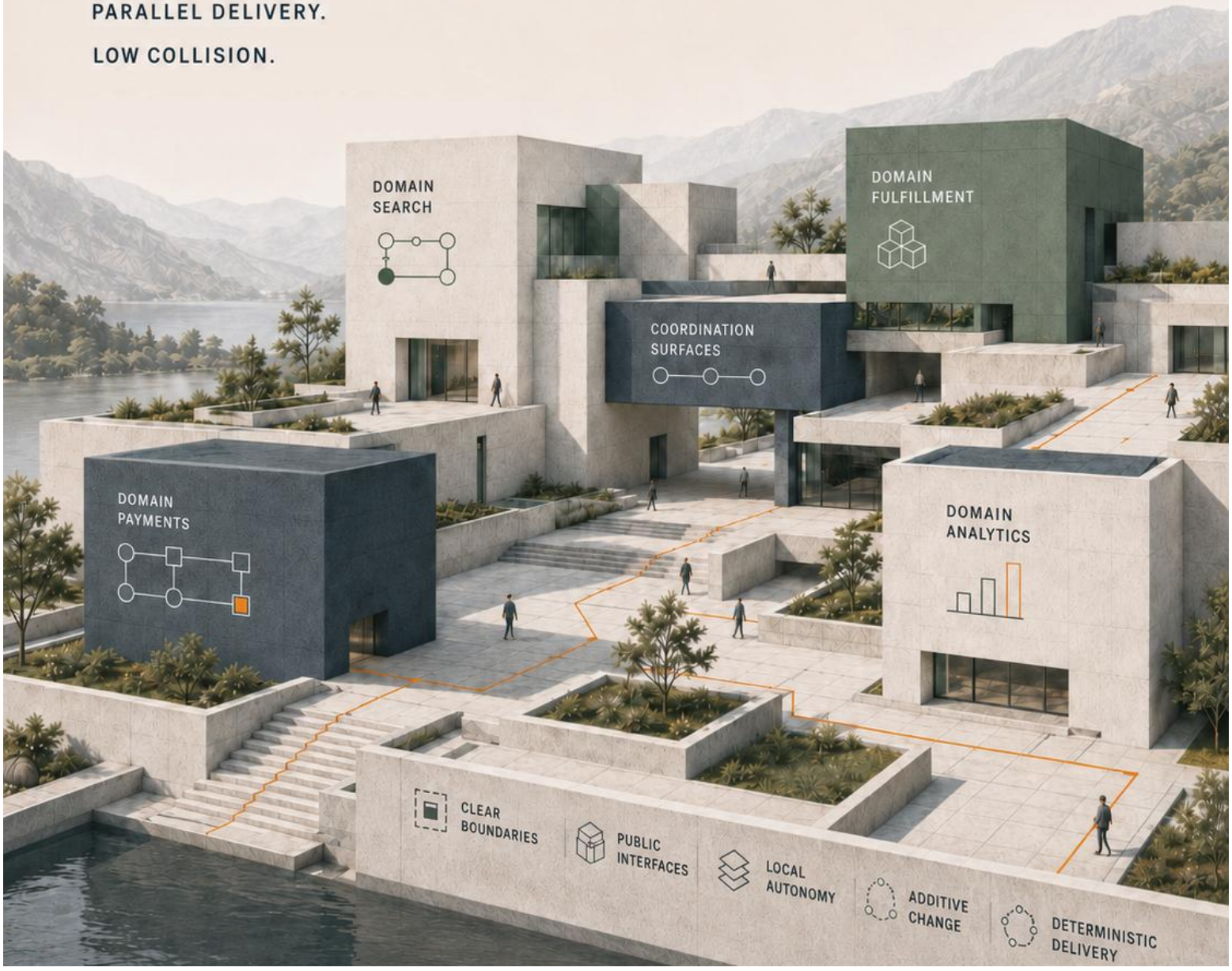
LOW COLLISION.

BOUNDED CONTEXTS

MODULAR MONOLITHS

STRUCTURED COORDINATION

ADDITIVE DELIVERY



WHITEPAPER

Contextrail Reference Architecture — Whitepaper

Version 0.8.0-4 | May 2026

Abstract

Contextrail is an open-source reference architecture template for building **Context-Optimized Architecture (COA) applications** in a world where AI agents are first-class citizens in agentic software development. Its core promise is not merely faster automation, but **cognitive ergonomics for agentic development**: code is partitioned into explicit hexagonal modules, cross-boundary work is constrained by additive seams, and sidecar metadata plus deterministic delivery rules keep context windows, token costs, and coordination overhead bounded as the system grows. The result is a repository shape that stays legible to both humans and agents, and that can support parallel delivery without turning every change into a full-repo reasoning problem.

«One repo. Many agents. Zero collisions.»

This whitepaper is written for technical leads evaluating adoption, senior engineers doing a deep technical read, and developers curious whether AI-native architecture holds up to scrutiny. It describes the parallel delivery model first (§6.5), the COA mechanism that makes small-context agents viable (§6.4), and the hexagonal structure that gives those mechanisms somewhere to stand (§3), backed by the operational model and the alternatives comparison (§5, §10). The claims here distinguish between **structural properties of the shipped template baseline**, reproducible measurements, and empirically validated workflows. Parallel multi-agent delivery is presented as a structural property of the architecture, not as a blanket promise that every downstream repository will preserve the same measurements automatically. Validation Status summarizes what is measured today and what remains intentionally out of scope for the public launch; §6.5 explains the mechanism and the measurements, and §10.2 separates structural properties from shipped features so the reader can judge each independently.

1. The Problem

AI coding agents are cheap to run and getting cheaper. The bottleneck is no longer "can one agent write code" but **"can N agents ship N features in parallel in the same repository"**. Typical codebases answer this badly for two specific reasons — and those are the reasons Contextrail exists.

Agents serialize on shared code. In a typical codebase, two agents working on two independent features will eventually touch the same shared utility, the same routing file, the same config, or the same UI layout. The result is merge conflicts, silent breakage, or wasted work — and the cost scales with the number of parallel agents. You do not get N× throughput from N agents; you get N× coordination overhead. No amount of prompt engineering or model capability fixes this, because the root cause is that the **repository structure** gives agents no structurally-safe place to add new behavior without touching code another agent already owns.

Agents waste context on neighbor-module internals. Even a single agent hits a hard ceiling: its context window. The moment the files it has to read to orient itself exceed that window, quality collapses — wrong files get touched, conventions drift, and the agent starts hallucinating APIs that do not exist. Typical codebases have no metadata layer that tells an agent what to skip, so agents deep-read entire modules just to answer the question "what is this?". This wastes the single most scarce resource in AI-assisted development — attention — on files the agent was never going to modify.

Contextrail is a direct answer to exactly these two problems. §6.5 explains how hex boundaries plus BBA-additive seams make parallel delivery structurally safe; §6.4 explains how COA lets a small-context agent orient on any module from a ~2.5K-token cold read without touching implementation code.

2. Design Philosophy

The template is built on five core principles:

2.1 Zero-Magic Architecture

Every module follows hexagonal architecture (ports and adapters) with a single public API entry point. There are no decorators, no dependency injection containers, no framework-level magic. Dependencies are explicit. Boundaries are enforced by contract tests. An agent — or a new team member — typically orients on a module through the tiered metadata chain: SYSTEM_MAP.md, manifest.json, public-api.mjs, README.md, and file-level headers or sidecars only where deeper precision is needed.

2.2 Browser-First and Framework-Agnostic

Hex modules run in any modern browser without Node.js built-ins and without a UI framework. Domain and port layers are pure ES modules that depend on nothing; adapters declare their own infrastructure dependencies only where the boundary actually crosses into infrastructure (the JWT adapter uses jose; the api-starter uses node:http). The practical consequence is that a small consuming application can ship without a bundler or transpiler — the vanilla and API starters do exactly that — while a React or Vue application can consume the same hex modules unchanged. The React starter uses Vite because that is the idiomatic React workflow; the modules it imports remain framework-free and bundler-free at the architecture layer.

This is a property that falls out of hex discipline, not a goal the template optimizes for. The point is that the domain slice of every module stays portable and inspectable; dependency minimalism is a side effect of that.

Two modules in the template make this claim sharper than any abstract argument can. The cqrs module implements command/query separation with a working event-sourcing path — the pattern most templates reach for NestJS or MediatR to provide — in twelve pure ES files with zero runtime dependencies and zero Node builtins. The graphql module implements a documented subset of the GraphQL query language, including its own tokenizer, parser, and executor, without importing graphql-js, Apollo, or any server framework. Both are patterns readers assume *require* a framework runtime; both demonstrate that the framework dependency is a habit, not a necessity. They exist in the template

as concrete proof that hex-pure domain code can carry patterns the industry treats as framework-only.

2.3 Process-as-Code

The delivery workflow is not documented in a wiki — it is encoded in deterministic scripts and git hooks. A pre-commit hook runs 17 distinct validation gates automatically (enumerated in §5.3) plus several auxiliary sync and repair passes (`spec-sync`, `backlog-sync`, `claim-check --auto-expire`, `claim-check --auto-complete`, `header-fix`, `readme-fix`, `agent-contract/sync`) that keep generated artifacts in step before the validation gates run: spec traceability, product docs consistency, USM coverage, architecture boundaries, header discipline, test gates, inter-agent claim conflicts, manifest capability drift, cross-harness adapter parity, changelog alignment, and changeset size limits. The hook supports tiered execution: `COA_GATE=fast` runs architecture + tests only for development speed, a docs-only fast path auto-detects documentation-only changesets, and `COA_SKIP_GATES` allows targeted bypass when needed. The process cannot drift because it is executable.

2.4 AI-Native Design

COA is architectural guardrails for AI agents. Where instruction files (`.cursorrules`, `CLAUDE.md`, `AGENTS.md`) are suggestions an agent can ignore, COA enforces correct behavior through structure: hex boundaries that make deep imports impossible, metadata that makes full-repo reads unnecessary, and deterministic gates that reject violations before they reach trunk. The right path is the easy path; the wrong path does not compile.

Every meaningful file carries a slim inline header plus an adjacent YAML sidecar (`<file>.header.md`) with hexagonal architecture fields (layer, port type, adapter type, bounded context, allowed/forbidden dependencies). This metadata enables AI agents to navigate the codebase without deep-reading implementation code.

2.5 Additive Modularity

The template ships with 40 hex modules in the current `v0.8.0` line, grouped here by role rather than enumerated flat:

- **Core platform primitives** — `state`, `event-bus`, `api-client`, `feature-seams` (BBA flags), `user-preferences`, `log`, `cache`, `i18n`, `notifications`, `form-validation`, `scheduler`, `task`, `permission`, `file`, `analytics`, `db`, `realtime`, `example-greeter`.

- **AI and retrieval stack** — retrieval (BM25 + vector hybrid RAG), knowledge-graph (entity/relationship extraction with BFS traversal), local-llm (in-browser WASM inference), ai-chat.
- **Auth and identity** — auth (JWT/OAuth pluggable), tenancy (multi-tenant context + isolation), user-management (registration, invitation, verification, password reset), subscription (plan tiers, entitlements, usage metering).
- **Server-side adapters** — monitoring (metrics, traces, health checks), job-queue (background jobs with retries), email (composition + transport), search (full-text with inverted index), payments (intent + webhook), rate-limit (token bucket / sliding window), openapi (document builder).
- **Delivery patterns and hex demonstrations** — cqrs (commands + queries + event sourcing, zero deps), graphql (schema + parser + executor, zero deps), onboarding, pwa (manifest + service-worker helpers), seo (meta tags + sitemap), theme (tokens + light/dark switching), prerender (static pre-rendering for crawlers).

retrieval and knowledge-graph together form the browser-side RAG stack; their pipeline architectures live in the respective module READMEs and are deliberately not repeated here. Each module can be safely detached via a CLI tool that reads dependency manifests, builds the reverse dependency graph, and removes the module along with its test files. Start with everything, remove what you don't need.

3. Architecture

3.1 Hexagonal Module Structure

Each module is a self-contained hex with three concentric layers and a single exported entry point:

Figure 1. Contextrail module anatomy: one legal entry and four downward dependency layers.

The on-disk shape mirrors the diagram:

```
modules/<name>/
├── domain/          Pure business logic, zero external dependencies
├── ports/           Contract definitions (assertXxxPort validators)
├── adapters/        Concrete implementations of port contracts
├── public-api.mjs   The ONLY cross-module entry point
├── messages.mjs     i18n strings for this module
├── types.d.ts       TypeScript definitions for tooling support
├── manifest.json    Dependency and metadata declaration
└── README.md        Module documentation
```

Meaningful source files may also carry adjacent `<file>.header.md` sidecars.

Key invariants:

- Cross-module imports go through `public-api.mjs` only — deep imports are forbidden
- Domain code has zero framework or infrastructure dependencies
- Ports define contracts; adapters implement them
- Multiple adapters per port (e.g., memory vs. persistent, echo vs. real)
- Contract tests automatically verify boundary compliance

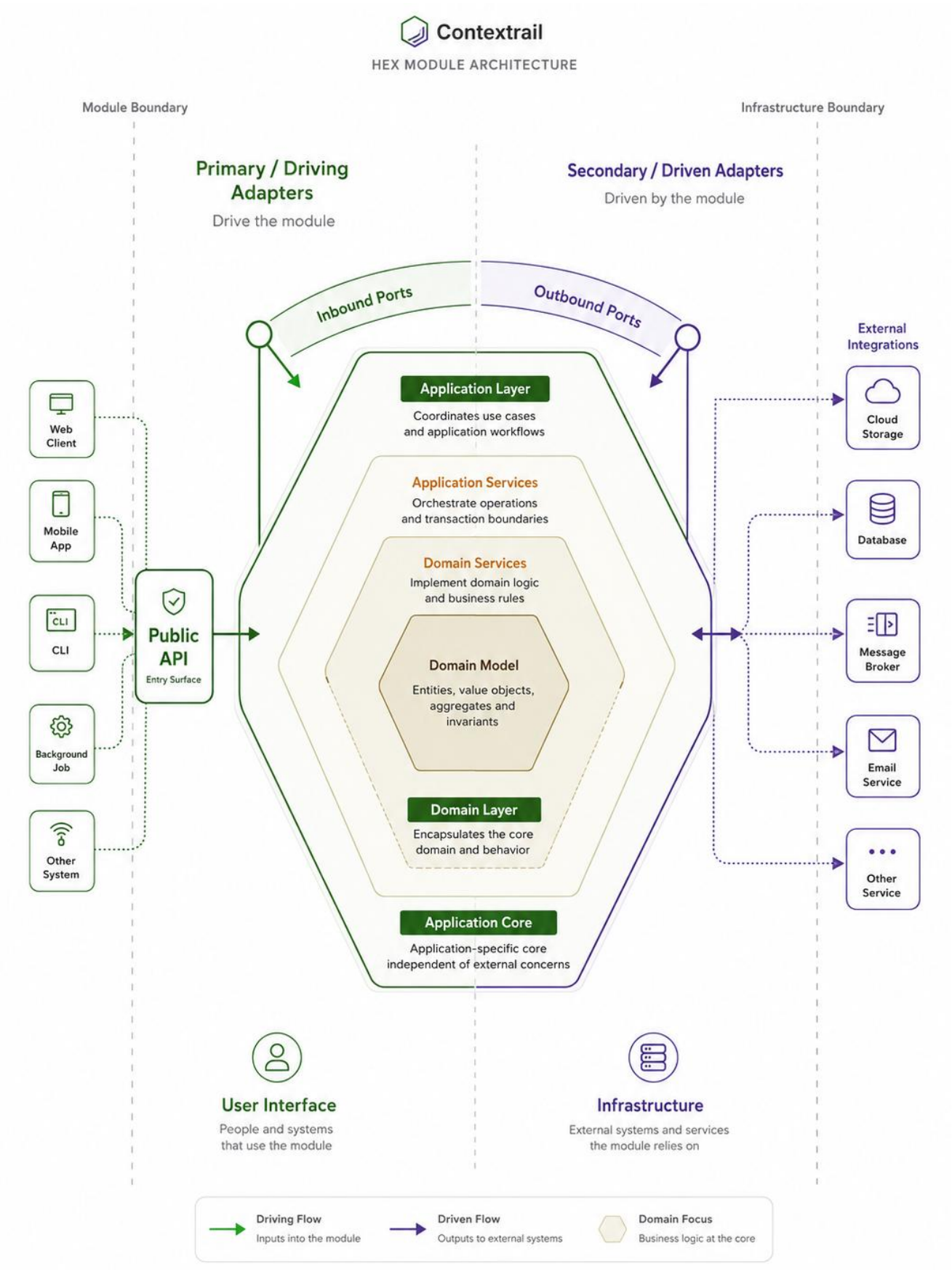


Figure 2. Hex module anatomy: public API, ports, adapters, and domain boundary.

3.2 Multi-Platform Abstraction

Three abstraction seams enable multiple runtime shells from one codebase:

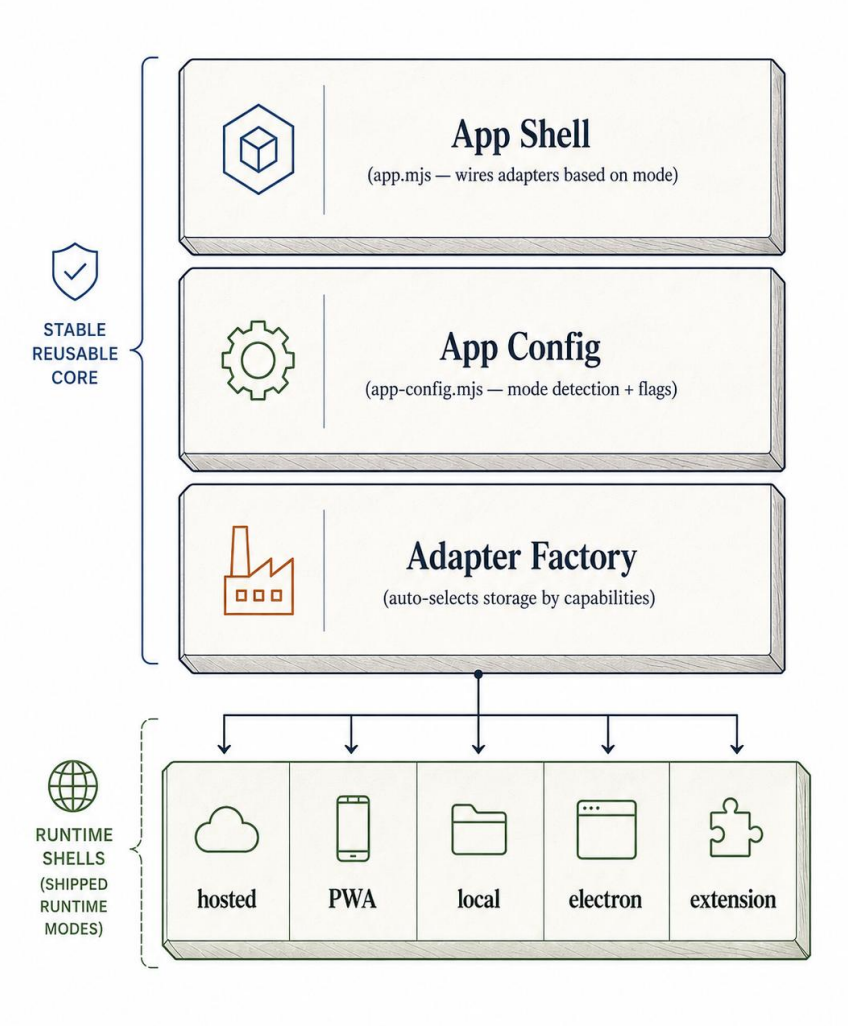


Figure 3. Runtime-shell layering: app shell, configuration, adapter factory, and shipped modes.

The current template ships five first-class modes in app-config.mjs: hosted, pwa, local, electron, and extension. Mode detection uses a priority chain: URL parameter → HTML meta tag → package.json default. Each mode configures which storage adapter (localStorage, IndexedDB, or memory) and which platform features (service worker, install prompt, offline cache) are active.[^capacitor]

[^capacitor]: A Capacitor scaffold also ships under templates/capacitor/ for mobile deployment, reusing the local mode with a native bridge. More broadly, the architecture is not limited to those five modes. They are the first-class modes currently shipped by the template. Additional runtime classes such as native-mobile wrappers, alternative desktop shells, embedded browser environments, or other platform-specific hosts can be added by introducing a suitable app shell and adapter layer.

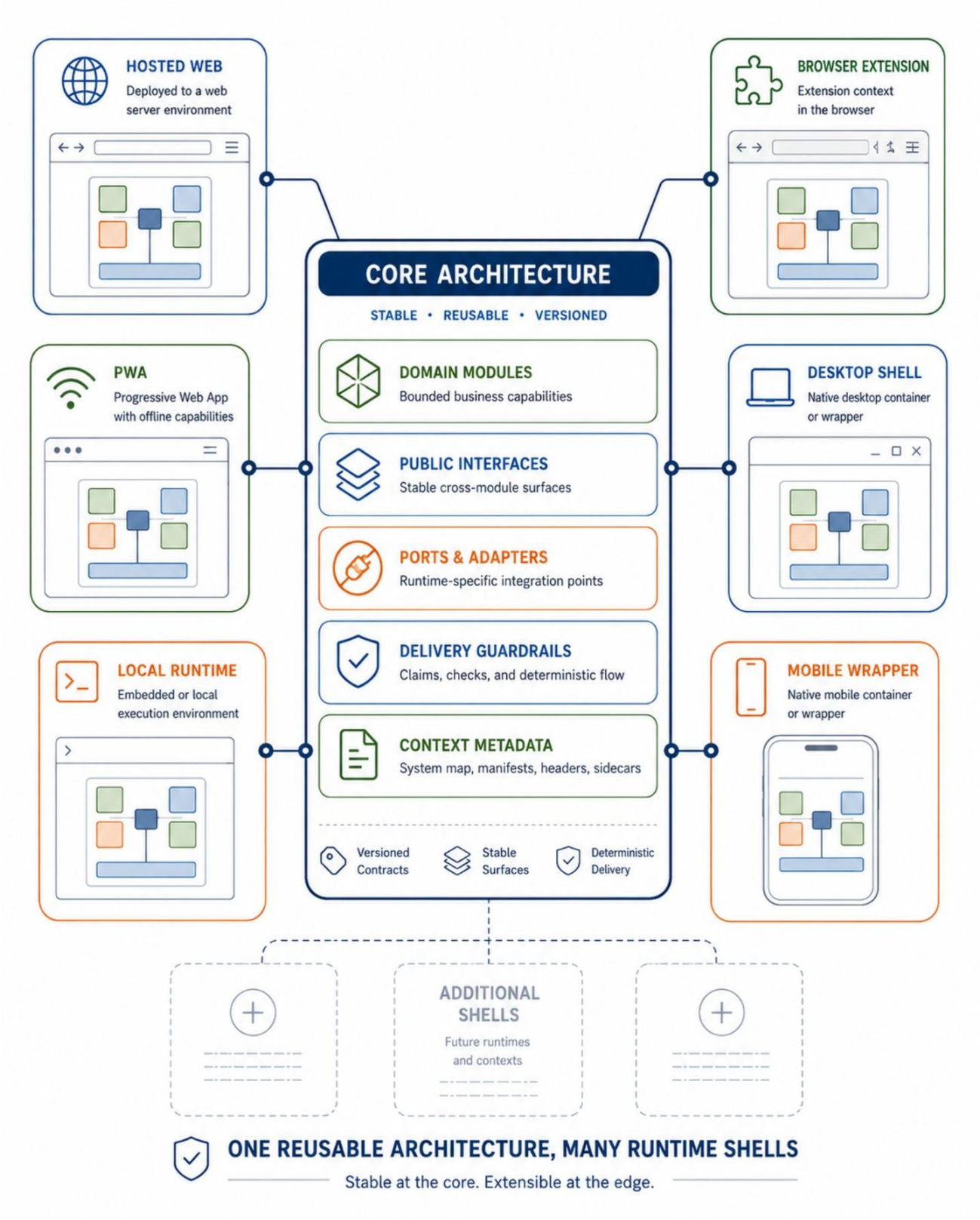


Figure 4. Core architecture reused across multiple runtime shells.

4. Language Strategy

The template uses **JavaScript + JSDoc + TypeScript declaration files** rather than TypeScript source code.

Rationale:

- Zero build step — browsers execute raw JS directly
- Native ESM — no module format conversion
- Runtime-inspectable — source code is the deployed code
- LLM-friendly — no transpilation artifacts or source maps to navigate
- Tooling support preserved — `.d.ts` sidecars provide autocomplete, navigation, and type-aware assistance

Trade-offs acknowledged:

- No TypeScript-source compile step; type intent is expressed through JSDoc, declaration files, and language-service tooling
- JSDoc verbosity for complex generics
- Some TypeScript-source ergonomics are traded away in exchange for zero-build execution and runtime-transparent source

If a downstream project later decides those ergonomics are worth the trade, the migration path is straightforward:

1. Rename module files incrementally from `.mjs` to `.ts`
2. Preserve the existing public API boundaries and manifests
3. Move JSDoc types into native TypeScript annotations where useful
4. Keep generated declaration surfaces only where external consumers still need them
5. Introduce a build step only at the point where the project actually benefits from it

In other words, Contextrail chooses JavaScript-first delivery by default; it does not trap adopters away from TypeScript later.

5. Delivery Model

5.1 Trunk-Based Development

The template uses trunk-based development with Branch by Abstraction (BBA):

- Trunk is the only long-lived branch
- New behavior is introduced behind feature seams (the `feature-seams` module)
- Old and new code paths coexist until the new path is proven green
- Transport branches live hours, not days

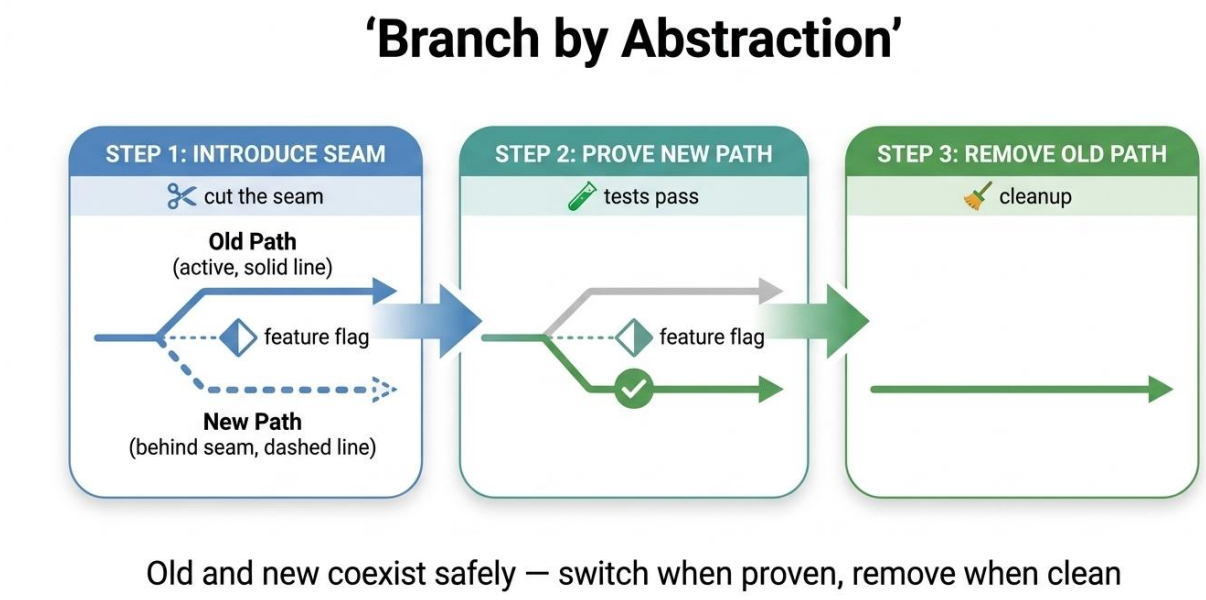


Figure 5. Branch by Abstraction: old and new paths coexist safely until the new path is proven and the old path can be removed.

5.2 Atomic Commits

One slice = one commit. A slice is the smallest independently reviewable, user-meaningful delivery unit, and it includes the full lifecycle needed to ship that change safely: tests, temporary seams or abstractions, implementation, documentation, traceability updates, and changelog coverage when those belong to the same atomic outcome.

5.3 Deterministic Quality Gates

The pre-commit hook runs automatically on every commit:

Gate	What it checks
spec-check	Traceability links resolve, no orphan IDs
product-docs-check	PRD/USM/backlog consistency
product-data-check	Persona and product-data integrity
usm-check	Persona and workflow coverage
design-docs-check	Design-lane routing and selector-registry wording
pre-impl-gate	Linked planning artifacts exist
claim-check	Active inter-agent claims and conflict enforcement
architecture-check	Hex boundaries, module structure
delivery-flow-check	Implementation/frontend/acceptance lane agreement
control-plane-check	Instructions/rules/scripts/hooks coherence
agent-contract:check	Generated adapter parity against the canonical compatibility contract
changeset-size-check	Changeset not too large
dependency-graph	Module dependency graph not stale
instruction-integrity-check	Canonical instruction files unchanged without CODEOWNERS review
capabilities-sync	Manifest capability surface matches public API
test-gate	All test layers pass
changelog-sync	CHANGELOG.md is current

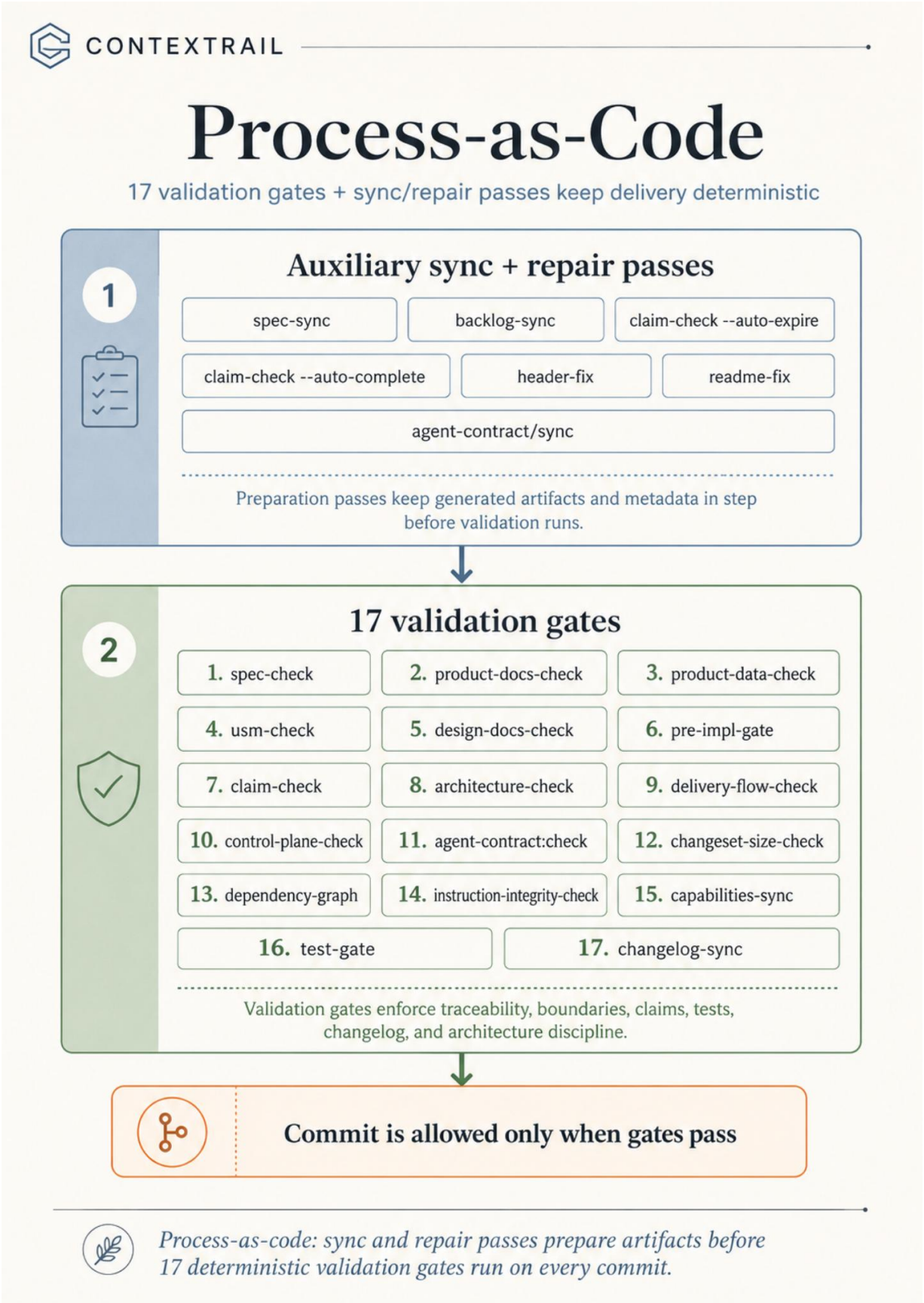


Figure 6. Process-as-code: sync and repair passes prepare artifacts before 17 deterministic validation gates run on every commit.

The hook also runs auxiliary sync and repair passes (`spec-sync`, `backlog-sync`, `header-fix`, `readme-fix`, `agent-contract/sync`, `claim-check --auto-expire/--auto-complete`) that keep generated artifacts in step before the validation gates run.

5.4 Traceability

A single ID namespace (`TPL-001`, `TPL-002`, ...) traces through the entire chain:

```
PRD (requirement intent)
  → USM (persona workflow)
    → Backlog (actionable slice)
      → Code headers (SpecRefs field)
        → Tests (SpecRefs in test description)
          → CHANGELOG (version entry)
            → Commit message (TPL-xxx references)
```

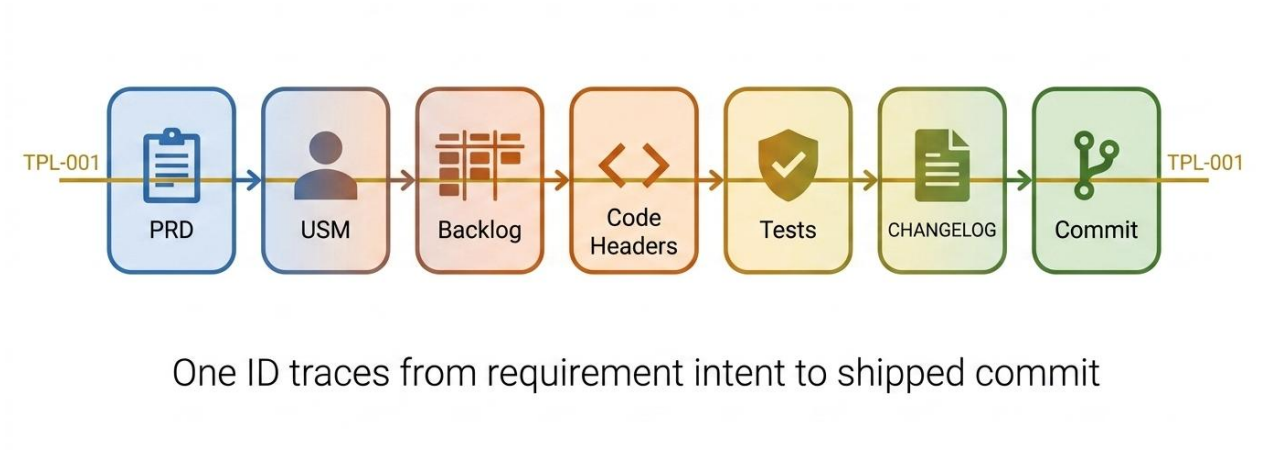


Figure 7. One traceability ID follows a slice from requirement intent to shipped commit.

6. AI-Assisted Development

6.1 Role-Specialized Delivery

The delivery model assumes role-specialized agent work, but the architecture does not depend on one fixed roster of named agents. What matters is the contract surface, bounded slices, and deterministic gates — not whether a particular harness exposes those roles through built-in agents, custom prompts, or future operator tooling.

6.2 Shared Compatibility Contract

Both Claude Code and OpenAI Codex operate from the same canonical contract (`docs/agent-contract/compatibility-contract.json`). Adapter files for each tool are generated from this single source:

- Claude: `.claude/CLAUDE.md`, `.claude/rules/`, `.claude/agents/`
- Codex: `AGENTS.md`, `.agents/`, `.agents/skills/`

Sync and verification:

```
node scripts/agent-contract/sync.mjs    # regenerate adapters
node scripts/agent-contract/check.mjs   # verify parity
```

6.3 Structured Headers for AI Navigation

Meaningful files carry a slim inline header plus an adjacent YAML sidecar (`<file>.header.md`) so AI agents can navigate without deep-reading source. The convention splits metadata into two surfaces: the inline block stays small enough that touching a file is cheap, while the sidecar holds the fuller machine-readable contract.

Slim inline header (in source):

```
/* @HEADER
 * @version 0.4.0 | 2026-04-06
 * @purpose BM25 keyword search adapter implementing RetrievalPort.
 * @sidecar bm25-adapter.mjs.header.md
 * @layer adapter | @hex outbound | @ctx retrieval
 * @public false
 * @edit careful
 */
```

Representative YAML sidecar fields (next to source):

```
---
fileId: contextrail-template:modules:retrieval:adapters:bm25-
adapter
hexLayer: adapter
adapterType: outbound
boundedContext: retrieval
implementsPort: RetrievalPort
summary: BM25 keyword search adapter for RetrievalPort.
allowedDependencies:
  - modules/retrieval/ports/*
  - modules/retrieval/domain/*
forbiddenDependencies:
  - modules/*/adapters/*
specRefs:
  - TPL-001
tests:
  - tests/unit/retrieval.test.mjs
related:
  - modules/retrieval/public-api.mjs
---
```

Both surfaces are machine-parseable and validated by `header-check.mjs`. Sidecars carry only fields with meaningful values — no `_none_` padding — so an agent reads exactly the metadata that exists rather than a fixed wall of empty placeholders.

6.4 Context-Optimized Architecture (COA)

The template is designed so that AI agents with **16K token context windows** — including local models running on consumer-grade GPUs — can perform structured software development without reading all source code. At the architectural level, 16K is the floor for general module work. At the empirical level, the repository's current local-LLM evidence covers **MICRO tasks and single-file LOCAL slices**; multi-file LOCAL delivery remains a roadmap item rather than a released promise.

Three pillars:

Pillar	Principle	Mechanism
Metadata over source	Agents read metadata, not implementation of neighboring modules	Structured headers, manifest.json, public-api.mjs, module READMEs
Boundaries = context walls	Hex module boundary = context boundary	Public-API barrel exports, forbidden deep imports
Atomic tasks = session scope	One agent session handles one bounded slice, then ends clean	Atomic slices, one-slice-one-commit, agent routing

Tiered documentation model:

Agents load documentation in tiers, stopping when they have enough context:

Tier	Content	Tokens	When
1	`docs/SYSTEM_MAP.md` — module index and dependency graph	~2204 full / ~992 focused	Always first
2	Module `manifest.json` + `public-api.mjs` + `README.md`	860–4305	For target module
3	`docs/module-catalog.md` section	~400	Only if full API needed

All figures below are measured by `bytes ÷ 4` against the **v0.8.0** tree via `scripts/demo/context-budget-demo.mjs`. Tier 2 varies widely by module complexity: `form-validation` is ~860 tokens; `auth` is ~2732; `retrieval` (the largest) is ~4305. For larger modules the capabilities surface remains a mitigation: agents can load `manifest.json#capabilities` alone (roughly ~600 tokens compact / ~1120 pretty-printed for `auth`) when they only need the public surface rather than the full manifest.

Three reproducible sample scenarios from the current demo:

1. Add a new OAuth provider to auth
Naive: 17,822 tok | COA: 4,224 tok | Saving: 76%

2. Wire an auth check into permission
Naive: 23,586 tok | COA: 6,577 tok | Saving: 72%

3. Fix a bug in a small core module
Naive: ~2,466 tok | COA: ~1,852 tok | COA value here is precise orientation: finding the right edit surface, boundaries, and nearby proof files without scanning unrelated implementation

16K is a practical floor for general module work in this template. The full instruction surface for Claude Code is ~6,150 tokens (CLAUDE.md + rules); external agents need ~400 tokens for a system prompt plus the full tiered loading chain. Smaller bounded tasks can fit below that, but larger modules and multi-module scenarios need more headroom, and the current public local-LLM evidence does **not** claim multi-file LOCAL support yet.

A **Context Loading Protocol** — a per-agent specification of which files to load, in what order, and at what measured token cost — is documented as an informational reference for orchestrators (see Context Loading Protocol). Agents orient on any module via the tiered model above, and the SYSTEM_MAP.md + capabilities surface together give a good-enough default loading order for most slices.

Key artifacts:

- **`docs/SYSTEM_MAP.md`** — ultra-compact system entry point (~2204 tokens full, ~992 focused)
- **`docs/_generated/dependency-graph.json`** — auto-generated forward/reverse dependencies, consumer edges, topological layers, safe removal order
- **`*modules/*/manifest.json`** — structure field lists actual file names per hex layer; dependents` field lists reverse dependencies
- **Summary field** in every file header — one-line description (~10-30 tokens) for quick scanning
- **`scripts/checks/dependency-graph.mjs`** — generator with --check mode for CI staleness detection

What happens when a module outgrows its context budget? COA treats this as a structural signal, not an edge case to work around. If an agent repeatedly

cannot orient on a module within the tiered loading budget, that is usually a sign the module should be decomposed. Larger context windows can help tactically, but they do not remove the architectural value of bounded modules. The repository also enforces soft size limits (180 lines for domain/ports, 400 for adapters) and `architecture-check.mjs` flags violations. In practice, a module that exceeds its Tier 2 budget (~2700 tokens at the p90) is a module with too many responsibilities — it should be split into two bounded contexts that each fit comfortably. This makes the context window a design constraint on the same level as memory or latency: when orientation pressure is chronic, the better long-term fix is often structural rather than simply increasing context. For brownfield migration, where a legacy file genuinely cannot be decomposed in one step, the sidecar header serves as a navigational map of the file's sections, and BBA seams allow incremental extraction slice by slice without rewriting the whole file at once.

A synthetic benchmark (`scripts/demo/context-budget-demo.mjs --core-only`) measures the difference across the 39 core modules of the shipped template: naive loading (read all source files) averages ~6,600 tokens per module; COA Tier 2 (manifest + public-api + README) averages ~1,878 tokens — a **73% reduction in Tier-2 context load** for large modules (≥6K naive) and **66%** for medium modules (3K–6K). Small modules often already fit in context; COA's value there is still navigational precision: agents can identify the correct edit surface, boundaries, tests, and related files with less ambiguity and fewer wrong turns. The core-module source total is ~257,393 tokens — no single agent can read it all, but any agent can orient on any core module for ~2,870 tokens (SYSTEM_MAP focused + average Tier 2).

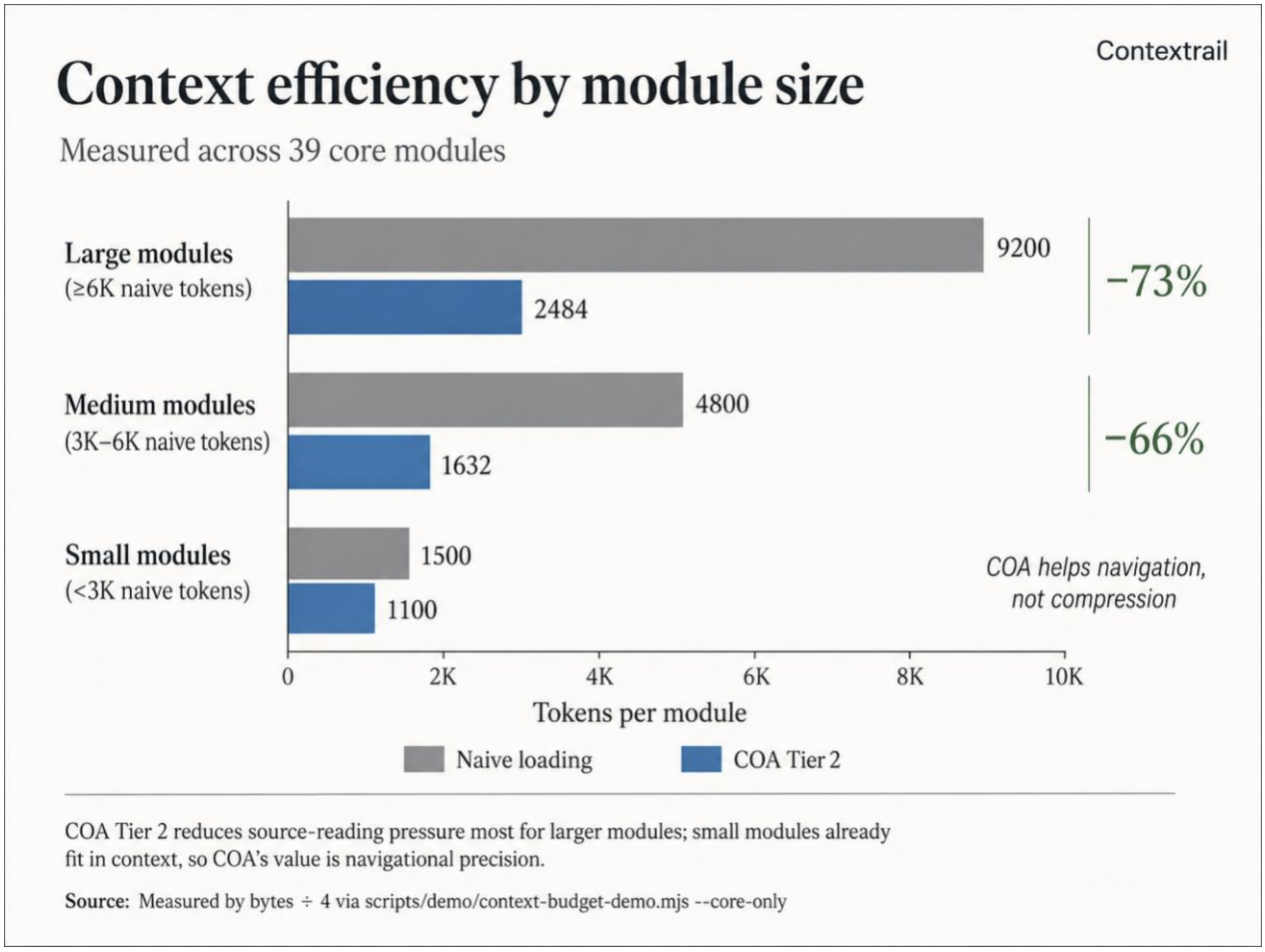


Figure 8. Context efficiency by module size: Tier-2 metadata loading reduces source-reading pressure most for larger modules.

This architecture democratizes AI-assisted development: consumer GPUs become sufficient for structured development, cloud models are reserved for complex planning, and the constraint of fitting every task in a bounded context window naturally produces small, bounded, well-documented modules.

See Context Loading Protocol for the full operational reference and the context budget demo for reproducible measurements.

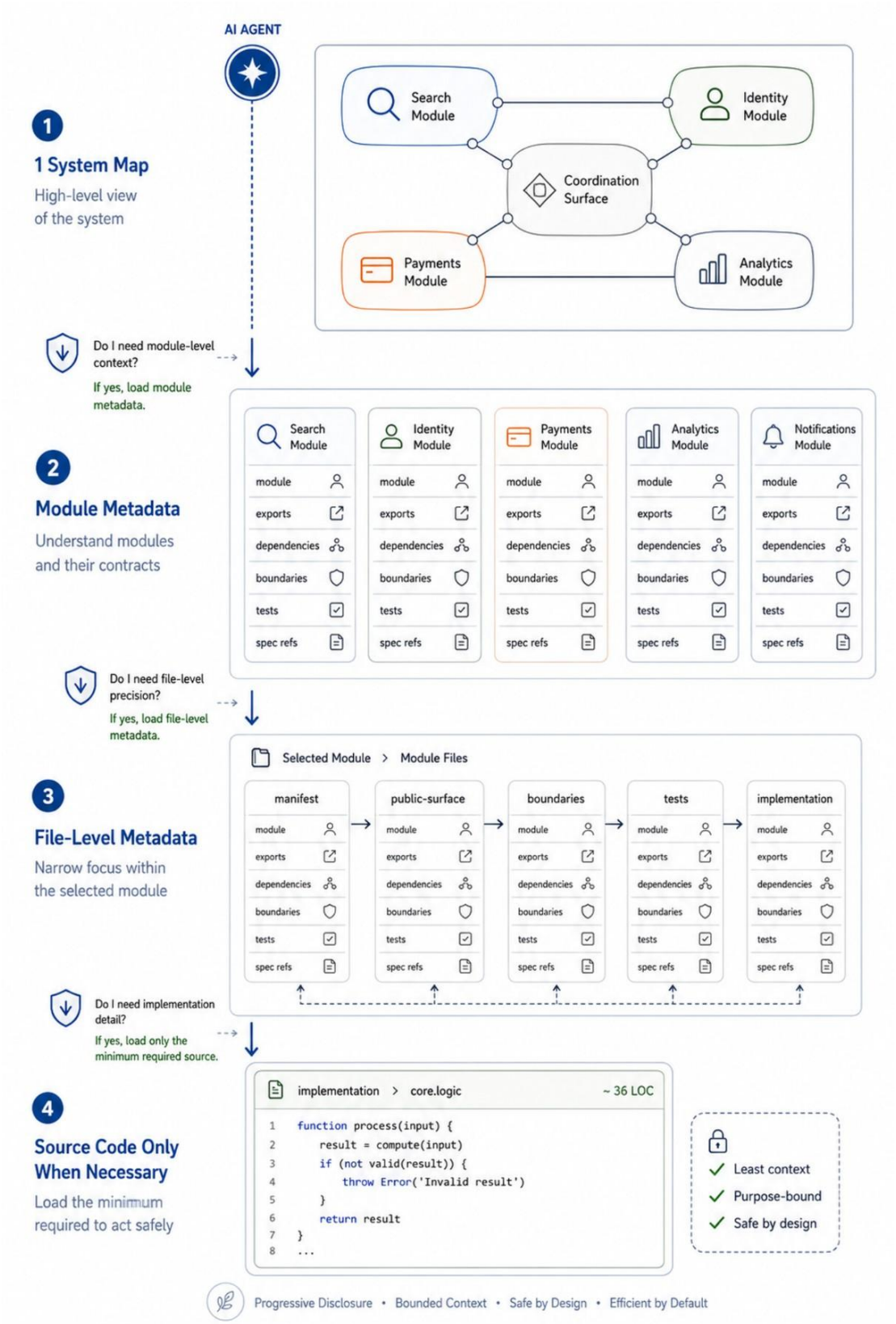


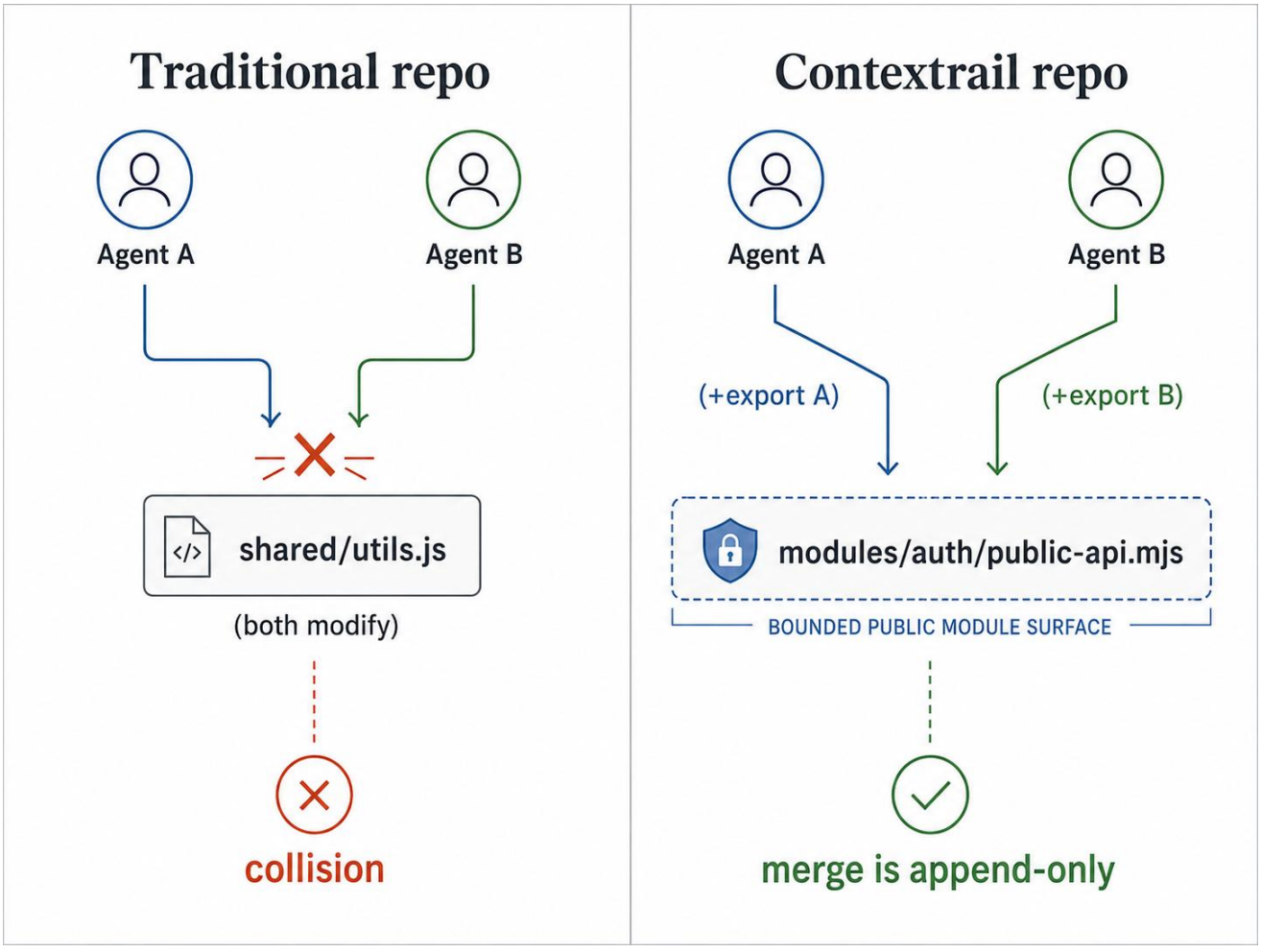
Figure 9. Progressive disclosure from system map to source code only when necessary.

6.5 Parallel Multiteam Delivery

COA's context efficiency is half the story. The other half is **parallel safety** — the architectural guarantee that multiple agent teams can deliver different features simultaneously in the same repository without merge conflicts or coordination overhead.

Why traditional repos serialize agent work:

In a typical codebase, two agents working on two features will eventually touch the same shared utility, the same configuration file, or the same API surface. The result is merge conflicts, silent breakage, or wasted work — and the cost scales with the number of parallel agents.



How COA makes parallel delivery the default:

Defense layer	What it prevents
Hex module boundaries	Agents touching code outside their module
public-api.mjs barrel	Cross-module coupling to internals
BBA seams + feature-seams module	Modifications to active code paths
Claims protocol	Overlapping edits on residual shared surfaces
Atomic slices + fast commits	Long conflict windows
Structured headers	Misunderstanding file intent

The critical insight: **if cross-boundary change is framed as addition wherever possible, conflict surfaces shrink dramatically**. Two agents can both add new exports to the same public-api.mjs — git merges additions cleanly. Two agents cannot both modify the same existing function signature without a collision.

BBA-first rule: When an agent needs to change another module's public API, it must first attempt to add a new export behind a BBA seam rather than modify an existing one. Only when modification is unavoidable (bug fix, rename, breaking change) does coordination become necessary.

Worked example — two agents, one public API. Agent A is tasked with adding magic-link verification to the auth module. Agent B is tasked with adding session revocation to the same module. Both agents will touch modules/auth/public-api.mjs. Under BBA-first they each write a pure addition:

```
// Agent A's slice
export { verifyMagicLink } from "./application/verify-magic-
link.mjs";

// Agent B's slice
export { revokeAllSessions } from "./application/revoke-all-
sessions.mjs";
```

Each slice lands as its own atomic commit. The merged public-api.mjs contains both new exports without either agent ever modifying a line the other wrote — git resolves append-only additions trivially, and neither slice can break the other's contract tests because neither touched existing exports. This is what "parallel-safe by construction" means in practice: the architecture reduces

shared-edit pressure by default, while claims, gates, and atomic-commit discipline guard the residual cases where shared modification is unavoidable.

Inter-agent coordination protocol: For the residual cases where cross-boundary modification is unavoidable — renaming an existing exported function, removing a port contract, changing a signature — a lightweight file-based claims mechanism in `.claims/` provides enforced coordination. Claims are structured JSON files that declare intent to modify a shared surface, enabling agents to detect overlaps before starting work. Claims now support pessimistic locking (`--acquire` mode) for fail-fast conflict detection at work-start, not just commit-time enforcement. The `--create` mode still works for backward compatibility.

```
{
  "id": "clm-022d3e",
  "agent": "feature-implementer",
  "slice": "TPL-319",
  "status": "active",
  "targets": [
    { "path": "docs/whitepaper.md", "action": "modify",
"surface": "shared-infra" }
  ],
  "strategy": "bba-additive",
  "priority": "medium"
}
```

```
# Recommended: acquire a pessimistic lock before starting work
node scripts/checks/claim-check.mjs --acquire \
  --agent=agent-a --slice=TPL-099 \
  --targets=modules/auth/public-api.mjs --action=modify
# → creates claim + lock atomically; fails if conflict exists

# At commit time, enforce blocks on active conflicts
node scripts/checks/claim-check.mjs --enforce --staged
# → conflict: modules/auth/public-api.mjs claimed by agent-a
(TPL-099)
```

A pre-commit hook blocks commits on active modify/replace conflicts and auto-expires stale claims. Protected shared-infra paths (`package.json`, `SYSTEM_MAP.md`, CI configs) produce advisory warnings when staged without a claim. The protocol degrades gracefully: ignoring it doesn't break anything that

hex boundaries and BBA don't already protect — it only adds a safety net for the slice of cases BBA cannot make additive.

The architectural goal is that many agent teams can deliver many features in parallel with sharply reduced coordination pressure. Each team leaves its own contextrail through the codebase, and the control plane exists to keep those trails legible and minimally overlapping.

Measured parallel capacity. A reproducible benchmark (`scripts/demo/context-budget-demo.mjs --parallel --core-only`) quantifies the claim against the live repository's 39 core modules, excluding the single teaching example. Of 741 possible core-module pairs, **734 (99%) are parallel-safe** — neither module depends on the other, so two agents can work on them simultaneously with zero coordination. The maximum independent set is **36 of 39 core modules**: 34 core modules sit at topological layer 0; only 5 core modules have any dependency edge at all (3 at depth 1, 2 at depth 2). The dependency graph is exceptionally shallow (max depth 2), and the 4 coupling hotspots — `api-client` (fan-in 2), `auth`, `ai-chat`, `user-preferences` (fan-in 1 each) — are the only modules where a change could require cross-agent awareness. Module independence scores reinforce this: 87% of core modules have zero dependencies, 90% are freely detachable with zero cascade, and the average self-containment is 94%. These numbers are a structural consequence of hex boundaries, not a coincidence — and they hold for any repository that follows the same module discipline.

Multi-module atomic slices. At the architecture level, 16K+ agents can hold context for 2-3 modules simultaneously, enabling cross-module atomic commits. This is legitimate when:

- A port contract evolves and consumers must update in the same commit
- A coordinated feature seam spans two modules that cannot land independently
- A shared type or enum extension must reach all consuming modules atomically

It is NOT legitimate for "while I'm here" cleanup or unrelated multi-feature batches. The litmus test: **would splitting into two commits leave the repo in a broken intermediate state?** If yes, multi-module is correct. If no, split.

Constraints: maximum 3 modules per atomic commit; claims must cover all targets before work starts; commit message must list all affected modules (`feat(auth,permission): ...`). Multi-module slices are estimated at ~10-15% of total work — if this percentage exceeds ~30%, the module boundaries may need restructuring.

This section describes the **structural delivery model**. The current BYO-LLM evidence does **not** yet claim empirical validation of multi-file LOCAL slices; that remains future validation work rather than a current release claim.

See Inter-Agent Coordination Protocol for the full protocol specification and the architecture demo for reproducible parallel capacity and module independence measurements.

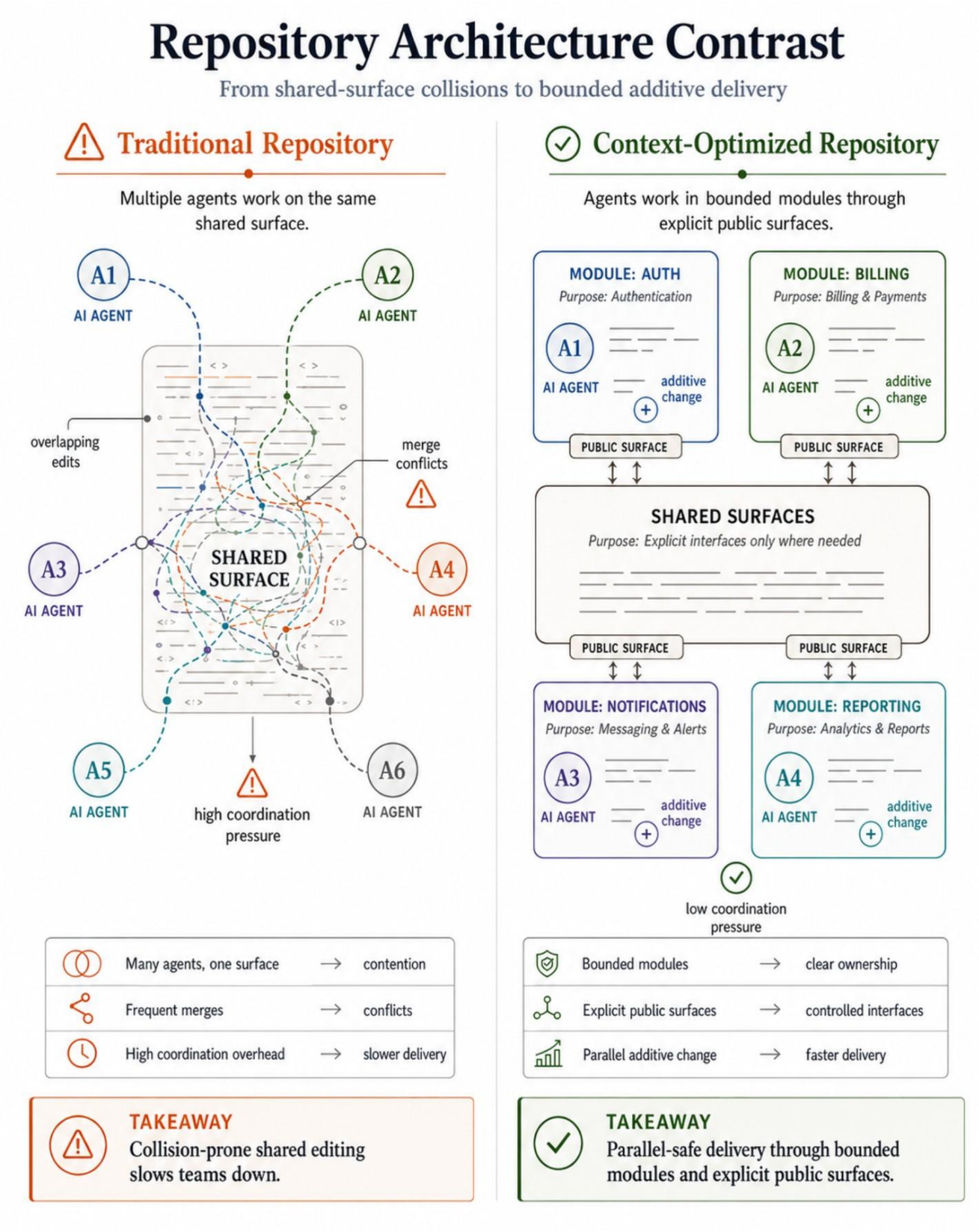


Figure 10. Repository architecture contrast between shared-surface collisions and bounded additive delivery.

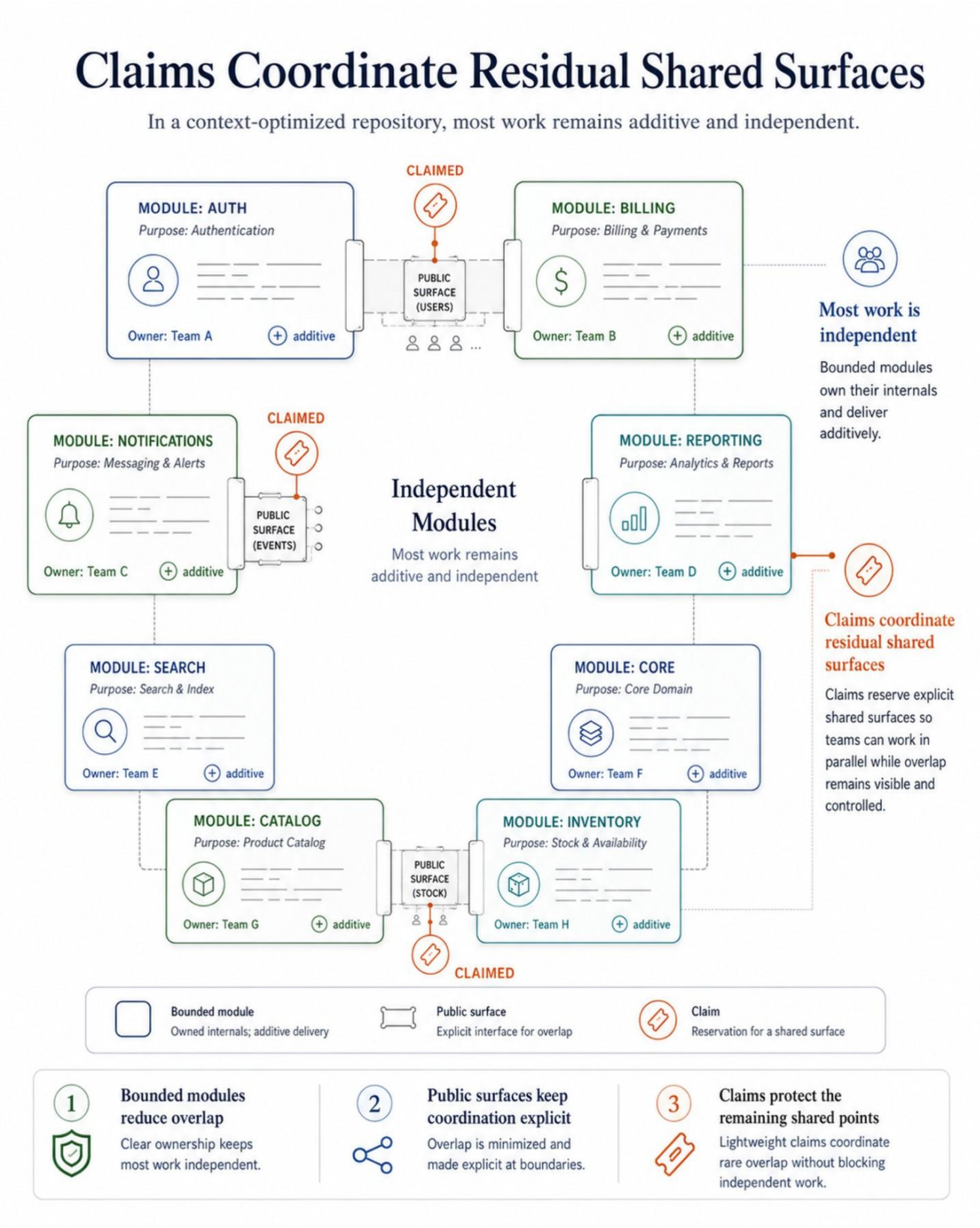


Figure 11. Claims used to coordinate residual shared public surfaces.

7. Testing Strategy

Five test layers provide comprehensive coverage. Exact layer counts evolve release to release, so this whitepaper names the layers and their purpose rather than freezing per-layer numbers that drift quickly.

Layer	Purpose	Runner
Unit	Pure logic, no I/O	`node --test`
Integration	System-level coherence	`node --test`
Contract	Hex boundary enforcement	`node --test`
BDD	Gherkin Given/When/Then	`node --test`
E2E	Browser smoke (optional)	Playwright

The repository's public landing surfaces (`README.md` and `docs/SYSTEM_MAP.md`) carry the current aggregate proof counts for the release line; the whitepaper focuses on the testing model rather than acting as a second mutable source of exact counts.

Key principles:

- TDD by default — tests written before implementation
- Bugfixes start with a failing regression test
- UI/UX changes require Gherkin scenarios
- Contract tests verify hex module boundaries automatically
- All tests run on Node.js built-in test runner — zero test framework dependencies

8. Design System

CSS architecture built around shared design tokens without preprocessors:

```
reset.css          → Modern CSS reset (element-only selectors)
  ↓
tokens.css         → Spacing, typography, shadows, z-index, width
  ↓
theme-variables.css → Colors, radius with light/dark auto-switching
  ↓
components.css     → Buttons, inputs, cards, badges, layout utilities
```

All component styles reference tokens — no raw hex colors or pixel values in component CSS. Dark mode works via `prefers-color-scheme` (automatic) and `data-theme="dark"` (manual toggle).

9. Framework Compatibility

9.1 Contextrail Is Not a UI Framework

A common first question: "Why not just use React?"

Contextrail and UI frameworks operate at different layers:

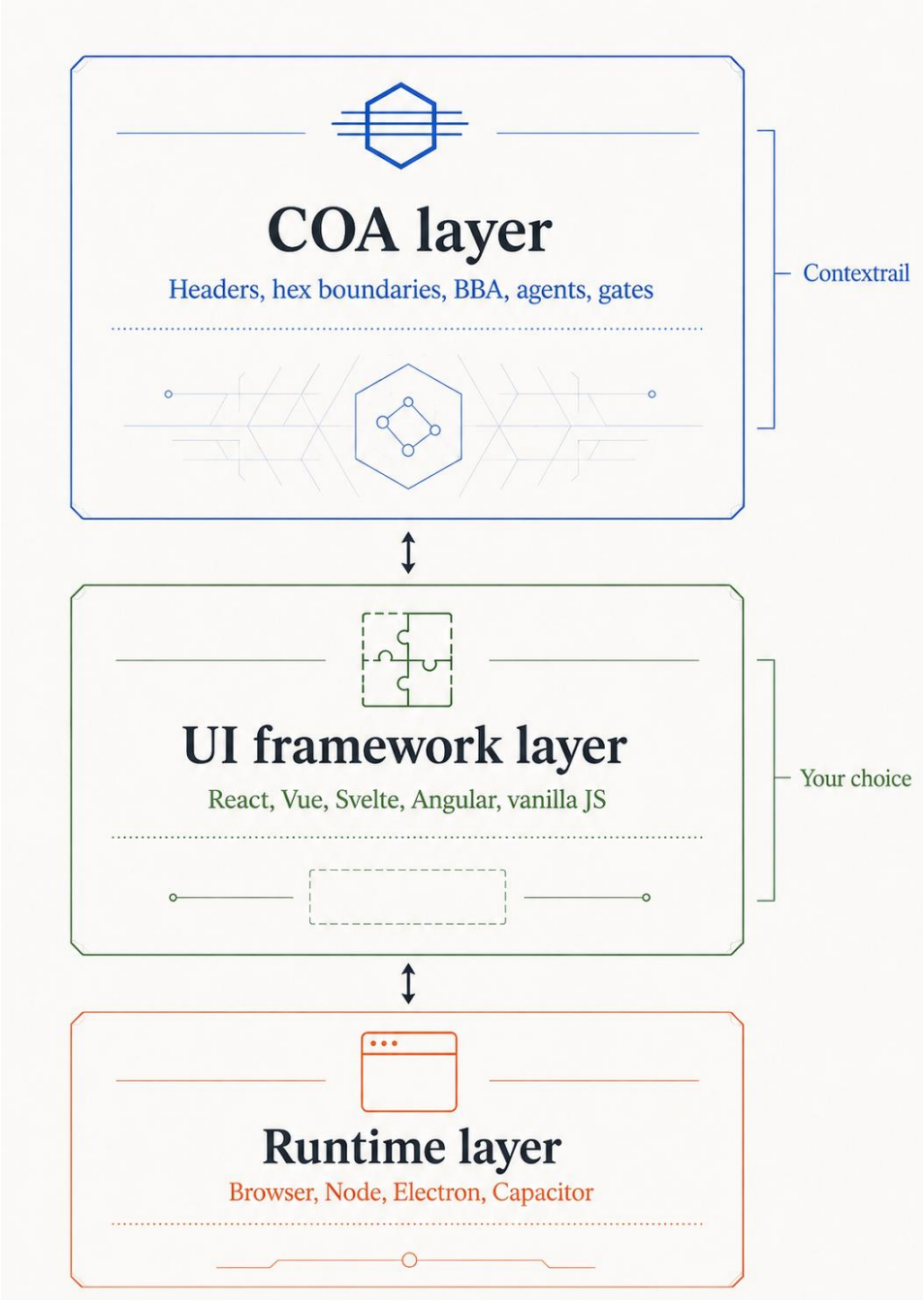


Figure 12. COA as the architectural control layer above replaceable UI frameworks and runtime environments.

These layers do not compete. React answers "how do I render UI." COA answers "how do I organize a repository so that N agents can work in parallel." A React project without COA is a repo where two agents cannot safely work simultaneously. COA without React is a repo with correct architecture but manual UI rendering.

The hex modules (`modules/*`) are pure ES modules with zero framework dependencies. Their domain logic, port contracts, and adapter interfaces work identically whether the consuming application is built with React, Vue, Svelte, Angular, or vanilla JavaScript.

9.2 The Adapter Bridge

Connecting a hex module to a UI framework requires one thin adapter per module — typically 10-20 lines. The pattern:

Framework	Adapter pattern	Example
React	Custom hook wrapping port adapter	`useNotifications()` calls `createNotification()` + React state
Vue	Composable wrapping port adapter	`useNotifications()` calls `createNotification()` + Vue `ref()`
Angular	Injectable service wrapping port adapter	`NotificationService` calls `createNotification()` + Angular `signal()`
Svelte	Store wrapping port adapter	`notifications` store calls `createNotification()` + Svelte `writable()`

Domain logic stays unchanged. Port contracts stay unchanged. Only the reactivity glue layer changes per framework.

9.3 Three Starter Apps

The template ships three starter applications demonstrating different integration approaches:

- **`apps/starter/`** — Vanilla JS. Zero framework dependencies. Shows the hex architecture in its purest form: direct DOM manipulation, manual event wiring, no build step. Best for understanding COA without framework noise.

- **`apps/react-starter/`** — React + Vite. Shows how the same hex modules integrate with a production UI framework. React hooks wrap hex adapters. Components consume domain logic through ports. The architecture layer (headers, boundaries, agents, gates) remains identical.
- **`apps/api-starter/`** — Node.js HTTP server with zero npm dependencies (`node:http` only). Shows hex modules wired on the server side — cache, log, event-bus, db — with JSON routing, CORS, and async error handling.

All three apps share the same `modules/` directory. The domain logic is written once; only the adapter layer differs per platform.

See Framework Integration Guide for step-by-step instructions covering React, Vue, Angular, and Svelte.

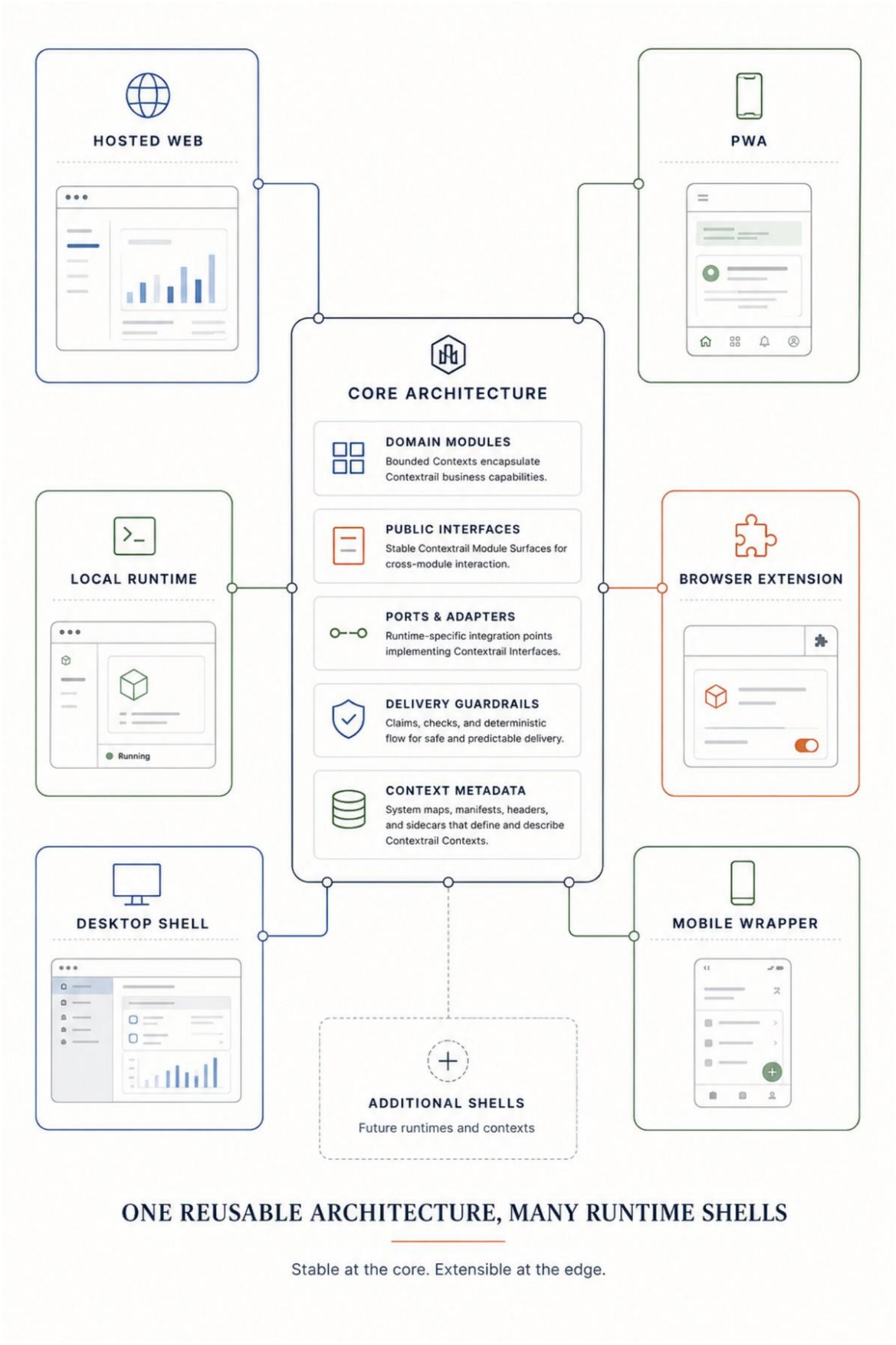


Figure 13. One reusable architecture serving multiple runtime shells.

10. Comparison with Alternatives

This section compares Contextrail against three popular starter stacks on two different axes: **shipped features** (capabilities you can check by cloning and running), and **architectural properties** (structural guarantees the architecture makes possible). Most "No" cells in the shipped-features table can be reached on Next.js or Vite by adding tooling, conventions, and discipline manually — Contextrail's claim is not that those stacks *cannot* do these things, but that they do not *ship* them as enforced defaults.

10.1 Shipped features (empirical)

These are things you can verify by cloning each repository and running the relevant commands. No judgement call required.

Feature	This Template	Create React App	Next.js	Vite Starter
Hex architecture enforced by gates	Yes	No	No	No
Framework-agnostic domain modules	Yes	No	No	No
Multi-platform from one codebase (5 targets)	Yes	No	Partial	No
Built-in RAG stack	Yes	No	No	No
AI agent orchestration (agents + skills + contract)	Yes	No	No	No
Deterministic quality gates on pre-commit	Yes	No	Partial[^1]	No
Module detachment CLI	Yes	No	No	No
Traceability (PRD→USM→backlog→code→test→commit)	Yes	No	No	No

[^1]: Next.js ships ESLint defaults and a build-time type check, but does not enforce architectural boundaries, traceability links, header discipline, or planning-artifact gates.

10.2 Architectural properties (structural)

These are **not** features you turn on — they are structural guarantees that follow from the architecture. They are the central claim of this whitepaper and the reason §6.5 exists. The parallel-delivery and context-efficiency rows are structural guarantees measured against the live repository (reproducible via scripts/demo/context-budget-demo.mjs), not an empirically validated production track record of N teams shipping simultaneously. The claim is that the architecture reduces conflict surfaces structurally, and that repositories following

the same module discipline can preserve the same qualitative advantages when the surrounding delivery rules are kept intact.

Property	This Template	Create React App	Next.js	Vite Starter
Parallel multi-agent delivery with sharply reduced coordination pressure — 99% parallel-safe pairs, 36/39 simultaneous core agents (§6.5)	Yes (measured)	No	No	No
Works with any UI framework in the adapter layer (see §9)	Yes	React only	React only	Any
Context-bounded module reads — 73% Tier-2 load reduction for large modules, ~2.8K tok orientation (§6.4)	Yes (measured)	No	No	No
Module independence — 87% zero-dep, 90% detachable, 94% self-contained (§6.5)	Yes (measured)	No	No	No

10.3 Where Contextrail is not a good fit

Contextrail is optimized for bounded agentic delivery in growing codebases. It is not the right default for every project shape. Consider alternatives when:

- **Your project is unlikely to grow past simple, low-coupling code.** If bounded agent navigation, modular isolation, and long-horizon maintainability are not important, the control plane may be unnecessary.
- **Your team does no AI-assisted development at all.** The parallel-delivery payoff disappears, and the template's other benefits (hex boundaries, deterministic gates) are available from simpler templates at lower cognitive cost.
- **Your project must be organized primarily around framework-owned structure.** Examples include heavily App-Router-centric Next.js codebases, Nuxt applications built around file-based conventions, or other stacks where the framework's directory model is intended to be the dominant architecture rather than a thin adapter shell.

10.4 Comparison with AI instruction surfaces

Several AI coding tools define their own project-level instruction formats. Contextrail's compatibility contract is the canonical source; per-tool adapters are generated from it.

Surface	Tool	Scope	Enforcement	Runtime blocking	Generated from contract
.claude/CLAUDE.md + .claude/rules/	Claude Code	Full project rules + routing	Loaded automatically	Yes (PreToolUse hooks)	Adapter
.cursorrules	Cursor	Project conventions	Loaded automatically	No	Yes (sync.mjs)
AGENTS.md + .agents/	Codex	Project contract + skills	Loaded automatically	No	Yes (sync.mjs)
.aider.conf.yml + CONVENTIONS.md	Aider	Project config + conventions	read: directive	No	Adapter
.windsurfrules	Windsurf	Project conventions	Loaded automatically	No	Not yet
CLAUDE.md (standalone)	Claude Code	Ad-hoc project notes	Loaded automatically	No	—

Key differences from standalone instruction files:

- **Single source of truth.** The compatibility contract (docs/agent-contract/compatibility-contract.json) defines principles, delivery flow, commands, roles, and skills once. sync.mjs generates tool-specific adapters. Editing .cursorrules or AGENTS.md directly is prohibited — changes go through the contract.
- **Deterministic enforcement.** Instruction files are suggestions. Contextrail backs them with architectural guardrails: pre-commit gates, architecture checks, and (for Claude) runtime command blocking. The wrong thing is not merely discouraged by convention; it is constrained by boundaries, checks, and control-plane enforcement.
- **Content validation.** The sync pipeline scans contract string fields for prompt-injection patterns (21 regexes). Standalone instruction files have no such protection.

11. Getting Started

```
# 1. Clone and install
git clone {{REPO_URL}} my-project
cd my-project
pnpm install

# 2. One-step bootstrap: replace placeholders + install git hooks
pnpm bootstrap -- --name "MyApp" --key "APP" --module "core"

# 3. Scaffold a new bounded hex module (optional)
pnpm create-module -- --name "payments" --description "Payment
processing"

# 4. Run tests
pnpm test

# 5. Build for your target platform
pnpm build:hosted    # Static site
pnpm build:pwa       # Progressive Web App
pnpm build:electron  # Desktop
pnpm build:local     # file:// protocol
pnpm build:extension # Browser extension (MV3)
```

All five declared modes from app-config.mjs (hosted, pwa, local, electron, extension) are now reachable through dedicated pnpm build:* scripts. The build:extension command assembles the templates/extension/ MV3 scaffold (manifest, background worker, popup) alongside the starter app output so the resulting dist/ is loadable unpacked in a Chromium browser.

pnpm bootstrap replaces the {{PROJECT_NAME}}, {{PROJECT_KEY}}, and {{DEFAULT_MODULE}} placeholders, updates package.json metadata, and installs git hooks in one step. The manual equivalent (placeholder replace + node scripts/checks/install-hooks.mjs) is documented in the README for adopters who want to see what bootstrap does.

See Getting Started Guide for the detailed walkthrough.

12. Roadmap

The repository is now in the **0.8.x** launch-preparation line. The roadmap is framed around hardening and additional validation rather than committed dates:

- **Current line (`0.8.x`)** — the open-core boundary is now explicit: Contextrail is cleanly scoped to the engine/language layer, while operator-facing UX is reserved for a separate launch. The manual BYO-LLM workflow remains documented for operators working directly from the repository.
- **Before 1.0** — Publication Readiness Audit pass, whitepaper/README hardening, sidecar referent cleanup, closure of remaining safety-guard edge cases, and additional empirical validation for broader local-LLM workflows such as multi-file LOCAL work.
- **Evidence roadmap** — the most important remaining proof item is not another feature module; it is a stronger public evidence pack around real operator onboarding, bounded local-LLM delivery, and launch-surface accuracy.
- **1.0** — stable release after the Publication Readiness Audit pass. **No committed timeline yet.**

Current Release Scope

The current release scope is intentionally bounded:

- **Pre-1.0 maturity** — Contextrail is in the 0.8.x public-launch line, and the stable 1.0 release follows a Publication Readiness Audit pass. There is no committed timeline yet.
- **Bounded local-LLM evidence** — public evidence today supports MICRO and single-file LOCAL slices, not arbitrary multi-file local development.
- **Selective tooling compatibility** — the documented BYO-LLM workflow is validated on selected stacks, not on every harness, model, or OS combination.
- **Engine-layer scope** — this launch covers the architecture template and manual workflow. The operator-facing product follows the open-core pattern as a separate launch on its own roadmap.
- **Hardening is still in progress** — some implementation details in the 0.8.x line remain advisory or are receiving follow-up fixes before 1.0, including metadata cleanup and completion of a few safety-guard edge cases.

13. Conclusion

Contextrail's central claim is stronger when phrased precisely:

- **Parallel-safe by construction** — the architecture is designed so that most work lands as additive change, with coordination reserved for the small set of genuine shared surfaces (§6.5).
- **Context-bounded by default** — agents do not need the whole repository in view to make bounded progress; the metadata surface is part of the architecture, not a documentation afterthought (§6.4).
- **Explicit about scope** — Contextrail is publishable as an engine-layer template before 1.0 because its current boundaries, evidence, and pending hardening work are clearly named (§12, §14).

14. Project Status

- **Version.** v0.8.0. Pre-1.0.
- **License.** Apache-2.0. Permissive — commercial use, modification, and redistribution are allowed with attribution.
- **How to file issues.** Bug reports, feature requests, and architectural questions: GitHub issues on the Contextrail repository. For security issues, please use GitHub's private vulnerability reporting channel rather than a public issue.
- **What is structurally validated.** The architecture demo and the deterministic gates validate the repository's current structure: 39 core modules plus 1 teaching example, 99% parallel-safe core-module pairs, 36 simultaneously editable core modules, shallow dependency depth, and bounded context surfaces reproducible from the live tree.
- **What is empirically validated.** The current BYO-LLM evidence covers MICRO-class work and single-file LOCAL slices on four measured stacks using the documented manual workflow. It does **not** yet claim multi-file LOCAL validation or a published N-team production case study.
- **Release timing.** The 1.0 release follows the Publication Readiness Audit pass — **no committed timeline yet**.

Validation Status

The public launch posture for Contextrail at **v0.8.0** is intentionally conservative:

- **Structurally measured and reproducible** — on the shipped v0.8.0 core-module baseline, the architecture demo reports 39 production modules plus 1 teaching example, with 734 of 741 core-module pairs as parallel-safe, 36 core modules editable without same-file contention, ~2.9K-token per-module orientation, and a 73% reduction in Tier-2 context load versus a naive full-source read for large modules. These are measurements of the shipped template structure, not a universal limit or guarantee for every downstream repository.
- **Empirically validated in bounded local-LLM work** — public evidence today supports **MICRO-class tasks and single-file LOCAL slices** on four BYO-LLM stacks using the documented manual workflow. This is evidence for bounded local use, not for arbitrary multi-file local development.
- **Still pre-1.0** — Contextrail is in the 0.8.x line, and the 1.0 release follows a Publication Readiness Audit pass. There is **no committed timeline yet**.
- **Open-core boundary is already in place** — Contextrail is the engine/language layer. An operator-facing product following the open-core pattern is planned as a separate launch.
- **Current release boundaries** — this release does not claim arbitrary multi-file local development, universal harness/model/OS compatibility, or complete hardening across every operational guardrail in the 0.8.x line.

Contextrail is an open-source reference architecture template and methodology kit built around Context-Optimized Architecture for parallel multiteam delivery. It is not affiliated with any AI model provider.